

Python Closures

- Python closure is a nested function that we can define a function from the inside of another function. And this function is called a nested function.
- A closure is a nested function that references one or more variables from its enclosing scope.
- function ke ander ke function ko nested function kahte hai.
-
- A closure is a function that **remembers** and **can access variables** from its enclosing scope even after the enclosing function has finished executing.
- Python ke nested-scope rules ke according, ek inner function apne enclosing function(bahar wale function ka scope) ke variables ko access kar sakta hai. Jab inner function ko return/pass kar diya jata hai aur woh enclosing scope ke variables ko remember karta rehta hai, usse closure kehte hain.

Example

```
def outer():          # Enclosing function
    x = 10

    def inner():      # Inner function
        print(x)

    return inner

func = outer()
func()

# Output:- 10
```

- Explain
 - outer() execute ho gaya aur normally uska local variable x local scope ke saath associated hota hai.
 - Lekin inner() ne x ko reference kiya tha, aur inner ko return kar diya gaya.
 - Isliye inner ke paas x ki value available rehti hai.
 - Yahi closure hai.

Note:-

- Ek important correction: enclosing ka matlab sirf "bahar" nahi, balki **nearest outer function scope hota hai**.
- Example

```
def outer():          # Outer function
    x = 10

    def middle():      # Outer ke andar
        x = 20

        def inner():   # Middle ke andar
            print(x)

        inner()

    middle()

outer()

# Output:- 20
```

- Agar middle() mein x na ho?

```
def outer():
    x = 10

    def middle():

        def inner():
            print(x)

        inner()

    middle()

outer()

# Output:- 10
```

Closure banne ke liye kya conditions hain?

- Generally closure ke liye 3 cheezein important hain:

1. Nested function hona chahiye:- Ek function ke andar doosra function:

```
def outer():  
    def inner():  
        pass
```

2. Inner function outer variable ko access kare

```
def outer():  
    x = 10  
  
    def inner():  
        print(x)
```

3. Inner function enclosing scope se bahar survive kare

```
return inner
```

Example

```
def outer():  
    x = 10  
  
    def inner():  
        print(x)  
  
    return inner  
  
func = outer()  
func()
```

- outer() khatam hone ke baad bhi func() x ko access kar sakta hai.

Nested Function vs Closure

- Nested Function** Sirf function ke andar function define karna:

```
def outer():  
  
    def inner():  
        print("Hello")  
  
    inner()
```

- Ye nested function hai. Lekin necessarily closure nahi.
- **Closure** Inner function enclosing function ke variable ko remember karta hai:

```
def outer():  
    x = 10  
  
    def inner():  
        print(x)  
  
    return inner
```

- Yahan inner x ko remember kar raha hai.

Note:-

- Every closure involves a nested function, but every nested function is not necessarily a closure.

```
def functionA():  
    print ("Outer function")  
    def functionB():  
        print ("Inner function")  
    functionB()
```

functionA()

Output:

Outer function

Inner function

```
def functionA(name):  
    print ("Outer function")  
    def functionB():  
        print ("Inner function")  
        print ("Hi {}".format(name))  
    # call inner function  
    functionB()
```

call outer function

```
functionA("Python")
```

Output:-

Outer function

Inner function

Hi Python